

Option Layers and Cuts

A scalable layer-propagation method for Sudoku, seven other logic puzzles and two optimisation problems — with experiments against classic algorithms

Shaimaa Said Soltan

September 2026

Keywords: Sudoku, constraint propagation, option layers, templates, cuts, MRV branching, exact cover, Dancing Links, Nonogram, Star Battle, Killer Sudoku, KenKen, N-Queens, Latin squares, graph colouring, branch and bound, Lagrangian bounds, large-neighbourhood search.

Contents

1. Summary
2. Background: the layer-propagation papers
3. The option-layer method
4. One method, eight puzzles
5. Solving experiments (including the lean engine, up to 30×30)
6. Optimisation: map colouring and Star Cost
7. Comparison with existing algorithms
8. When option layers win and when MRV wins
9. The cube as a tensor: three more engines
10. Discussion, limits and related work
11. Appendix: seeds, settings and how to reproduce

All figures in this report were produced from the author's offline HTML pages, run in a headless browser with fixed random seeds. Screenshots are taken from the same pages at phone size. Nothing was sent to a server.

1. Summary

This report describes and tests the **option-layer** method, an extension of the author's layer-propagation Sudoku solver. Each number's layer is described by the list of its whole placements ("options"). Options that cannot fit are **cut**; cells used by every option are **certain**; a layer with one option left is **locked**. Layers are built thinnest first, pairs and groups of 3–4 layers are checked together, and a guess is made only as the last step.

The same three ideas — **units, options, pillars** — were used for eight puzzles (Sudoku, Latin squares, Killer Sudoku, KenKen, Nonograms, Star Battle, N-Queens and map colouring) and for a new optimisation problem, **Star Cost**: Star Battle with a weight on every cell, where the cheapest placement is wanted.

1,066

logic puzzles of seven types, all solved correctly

98%

solved with no guess at all

500 / 500

9×9 Sudoku (22–40 givens) with no guess

2–160×

lean option layers faster than the full engine; fastest median from 9×9 to 30×30

Main findings.

- Option layers solved every 9×9 Sudoku without a guess; the 3/4-number group check was needed for 37 of them. MRV backtracking needed about 1,186 search nodes per puzzle at 22 givens.
- For N-Queens with givens (up to 30×30) and sparse Nonograms, option layers did clearly better than classic search: row backtracking failed on 5 boards within 5 million nodes, and a line solver guessed on 16 Nonograms.
- A **lean** version that keeps only build, cut and AND (starting from the top 2 layers) is 2–160× faster than the full engine — the gap grows with the board. Started from the numbers with the fewest open cells, it had the fastest median time of all methods on every board from 9×9 to 30×30, ahead of Dancing Links and MRV backtracking. It pays with some guesses, which the full engine avoids.
- For Star Cost, a Lagrangian ("weight descent") bound proves answers that the simple bound cannot. The author's lock-cheapest rule with look-ahead, followed by Unstick (a large-neighbourhood rebuild), came within about 5–6% of the best on large boards, ahead of simulated annealing and random-restart descent.

2. Background: the layer-propagation papers

This work continues three papers by the author, *A Scalable Layer-Propagation and MRV Branching Method for Solving and Generating Large $N \times N$ Sudoku* (versions 1–3). The papers describe a hybrid solver for any board size, from 4×4 up to 36×36, with rectangular boxes for sizes such as 6×6 (2×3 boxes) and 12×12 (3×4 boxes). It has five parts:

1. **Layer-based propagation.** The board is seen as N stacked layers, one per number. Layer d marks where d can still go. A placed number clears its row, column and box in its own layer, and clears its cell in every other layer.
2. **Single-missing deduction.** A row, column or box with one place left for a number gets it at once.

3. **MRV candidate selection.** When logic stops, the cell with the fewest candidates (minimum remaining values) is chosen.
4. **Shallow branching with forbidden-value learning.** A guess that fails is remembered as forbidden, so the same dead end is not tried twice.
5. **Early conflict detection**, and a **generator** that removes clues with symmetry and structure checks, so large puzzles stay solvable.

The papers report results for 9×9 to 36×36 at 25% and 35% clues, and compare the method with pure backtracking and human-style logic solvers.

The step described in this report grows out of the first idea: **if each number is a layer, then each layer can be described by the list of whole placements it could still take.** We call these placements *options*. Working with whole options, instead of single cells, lets the solver see facts that single-cell logic misses. It also carries over, almost unchanged, to other puzzles and to optimisation.

3. The option-layer method

3.1 Units, options and pillars

Every puzzle in this report is described with three words:

- **Unit.** A part of the puzzle that must be completed as a whole. In Sudoku, a unit is one number's layer: the nine places of the digit 5.
- **Option.** One complete way to fill a unit. For the digit 5 in a 9×9 Sudoku, an option is a set of nine cells with one cell in every row, column and box. (Such a set is also called a *template*.) At the start there are 46,656 options per digit; givens reduce this quickly.
- **Pillar.** The thing the units share. In Sudoku this is the cell: every cell must be used by exactly one digit's option.

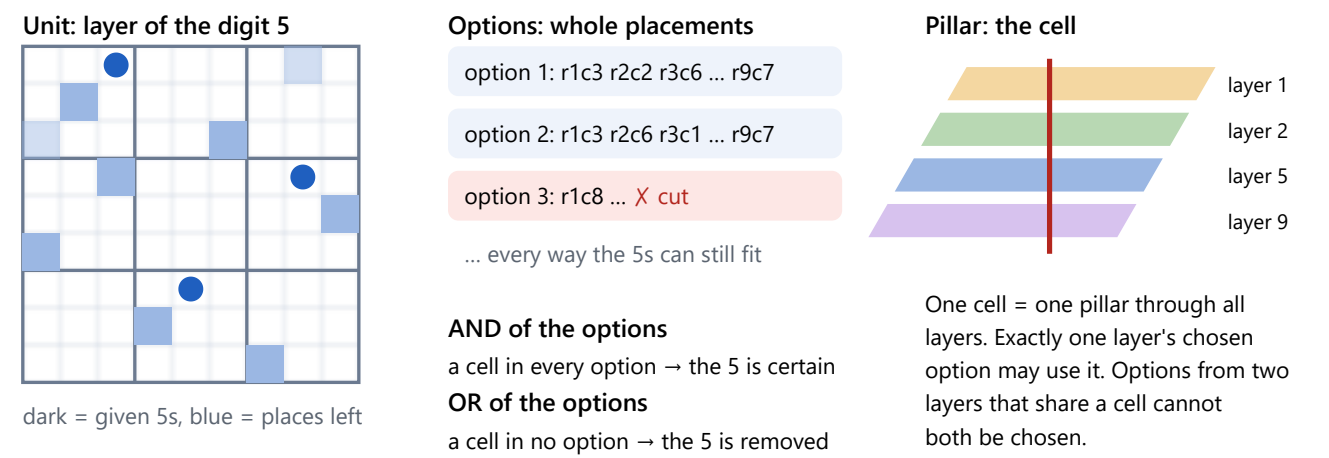


Figure 3.1. The three words of the method. Each unit keeps a list of whole options. The AND of its options gives certain cells; the OR gives the cells that are still possible.

3.2 The rules

The method repeats a few simple rules until the board is solved. Each rule is cheap to explain, and every step it takes can be shown to a person as a reason.

Rule	What it does
Cut	Remove every option that uses a cell which is already taken, or a cell whose candidate for this unit has been removed.
AND (certain cells)	If a cell is used by every remaining option of a unit, place it.
Narrow	If a cell is used by no remaining option of a unit, remove that unit's candidate from the cell. This is the OR of the options.
Singles	A cell with one candidate left, or a unit with one place left in a row, column or box, is filled (the papers' single-missing deduction).
 Lock	A unit with exactly one option left is locked: its whole placement is written at once.
Pair check	Two units that share cells must have options that fit together. An option of unit A that clashes with every option of unit B is cut.
Group check	The same test for 3 or 4 units together: an option is cut if no choice for the other units in the group fits beside it.
Probe (all units together)	Used in puzzles without a cheap group check: assume one option, run all the rules, and cut the option if a contradiction appears.
Guess	Only when nothing else works: try the smallest unit's options one by one, with backtracking (the papers' MRV branching).

3.3 Thinnest first: building layers one at a time

Listing every option of every unit can be expensive. The method therefore **builds option layers incrementally**. It starts with the two units that have the fewest options, and adds the next thinnest unit only when the checks stop making progress. When two units have the same count, the one with fewer free cells in a box goes first. On most puzzles the board is solved before the larger layers are ever listed; this is the "option layers built" count in Section 5.

```
function solveOptionLayers(board):
  units ← all units, sorted by estimated option count (fewest first)
  active ← first two units; build their option lists
  loop
    changed ← applyRules(active)           // cut, AND, narrow, singles, lock
    if board solved: return board
    if changed: continue
    if pairCheck(active) cut something: continue
    if groupCheck(active, size 3..4) cut something: continue
    if some unit not active: active ← active + nextThinnest(); continue
    if probe(active) cut something: continue // puzzles without a group check
    u ← unit with fewest options             // guess last (MRV)
    for each option o of u:
      if solveOptionLayers(board with o locked) succeeds: return it
  return fail
```

3.4 Why whole options see more

Single-cell logic looks at one cell or one line at a time. An option joins all nine places of a digit, so a single cut can reach across the whole board. Classic human techniques such as *X-Wing*, *Swordfish* and *Jellyfish* for one digit are special cases of the **AND/narrow** rule on one layer: they fall out automatically, without being coded. The pair check covers many two-digit patterns, and the 3/4-number group check covers the remaining hard cases in our tests (Section 5.1).

A guess-free explanation

Every step before the first guess is a reason that a person can check: "no option of the 7s uses this cell", "every option of the 3s uses this cell", "this option of the 2s leaves no room for any option of the 8s". The pages call these steps the **sure** part of the log, and the hint button only uses them.

4. One method, eight puzzles

The same three words fit many puzzles. Only the list of units, and the way options are listed, changes. The table below is the "translation" used by each page.

Puzzle	Units	Options	Pillar	Extra rule
Sudoku (4×4 to 16×16)	one layer per number	templates (one place per row, column, box)	cell	—
Latin square	one layer per number	permutations (no boxes)	cell	—
Killer Sudoku	number layers + cage layers	templates; digit sets that meet the cage sum	cell	no repeats in a cage
KenKen	number layers + cage layers	permutations; fillings that meet the cage target with +, −, ×, ÷	cell	—
Nonogram	rows and columns	every placement of the line's blocks	cell	—
Star Battle	rows, columns, regions (+ optional 3×3 boxes)	sets of K stars in the unit	cell	a star empties its 8 neighbours
N-Queens	rows and columns	one cell per unit	cell	a queen empties its diagonals
Map colouring	regions	colours	border	triangle group check

The rules of Section 3 run unchanged. For example, in a Nonogram the AND of a row's options is the classic "overlap" rule, and the pair check between a row and a column is the usual line-crossing logic. In Star Battle, the probe ("all units together") finds the placements that would squeeze a region to nothing.

4.1 Sudoku: the game and the cube view

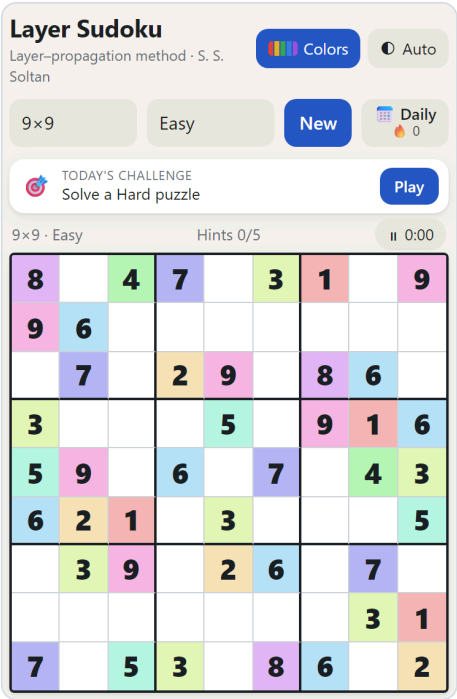


Figure 4.1. The main game page.

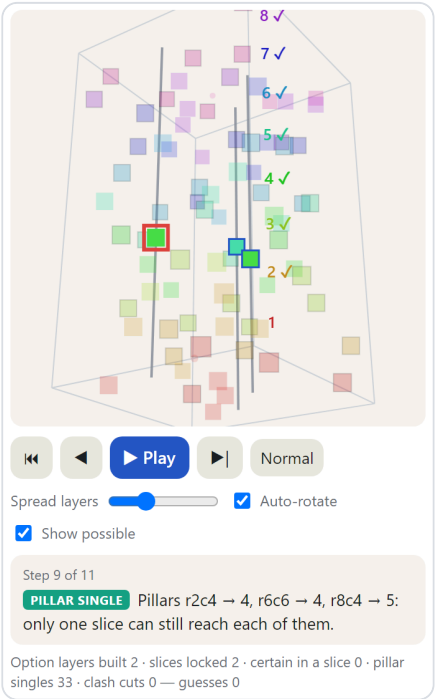


Figure 4.2. The same board as N stacked number layers (slice mode). A pillar single: only one layer can still reach a cell.

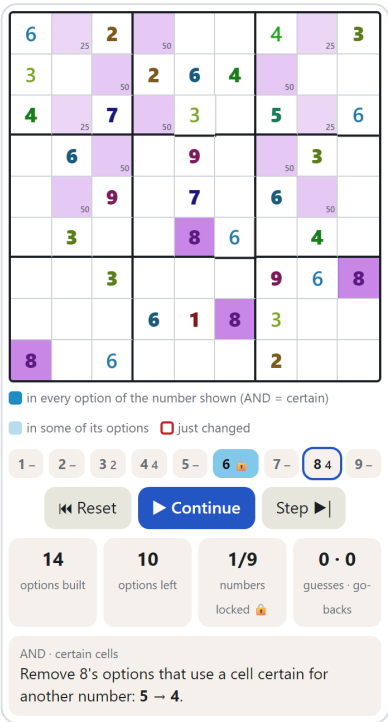


Figure 4.3. Option Layers page mid-solve. Shaded cells show the share (%) of the chosen number's options that use them; the chips below give the options left per number (🔒 = locked).

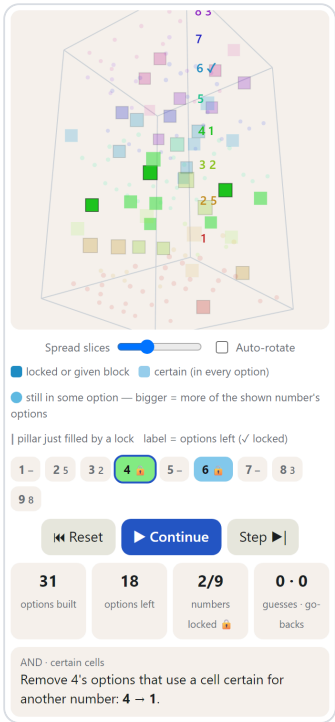


Figure 4.4. The option layers drawn as a cube: each slice is one number.

4.2 The other puzzles

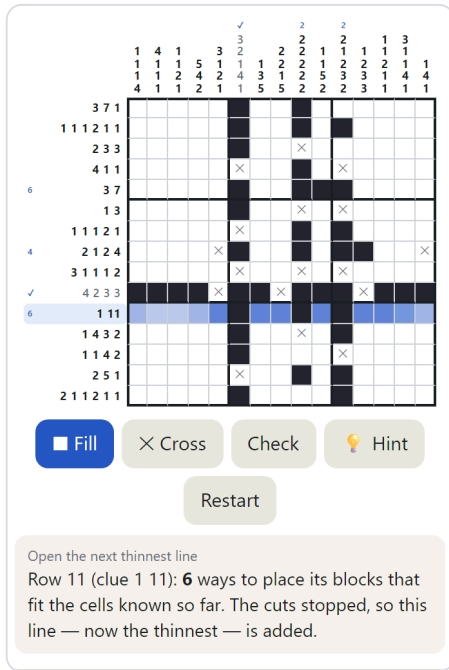


Figure 4.5. Nonogram: lines as units; certain cells come from the AND of each line's placements.

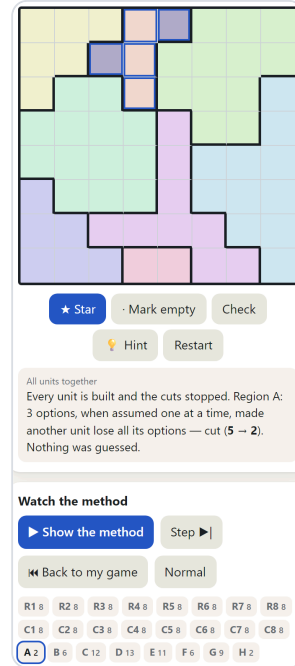


Figure 4.6. Star Battle: a probe step ("all units together") cutting an option.



Figure 4.7. Killer Sudoku: number layers and cage layers.

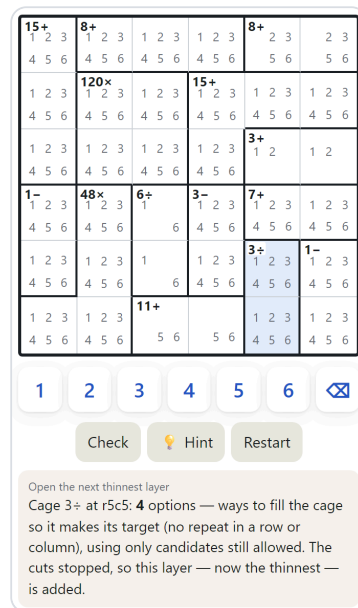


Figure 4.8. KenKen: cages carry an operation and a target.

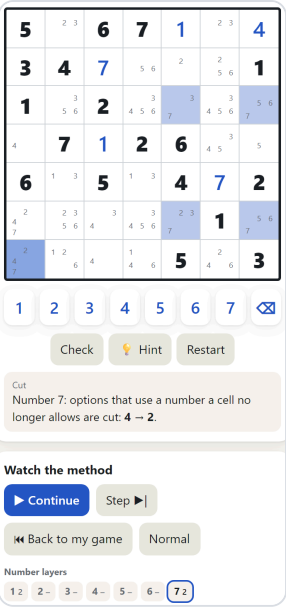


Figure 4.9. Latin square: Sudoku without boxes.

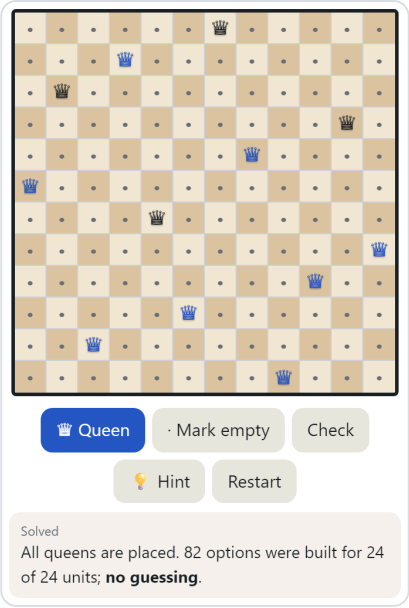


Figure 4.10. N-Queens 12×12, solved from the fewest givens that keep it unique.

5. Solving experiments

Set-up. Every page was loaded in a headless Microsoft Edge browser and driven by a small script (Appendix). The browser's random numbers were replaced by a seeded generator, so each puzzle can be made again from its seed. Every method ran on the *same* puzzles, one after the other, on one core. Every solution was checked against the rules. Times are browser milliseconds; the timer steps in units of 0.1 ms, so very small times are rounded.

Methods compared.

- **Option layers** — the method of Section 3, as used in the pages.
- **Option layers, pairs only** — the same without the 3/4-number group check, to measure what that check adds.
- **Layer Builder** — the author's earlier layer-propagation solver with MRV branching (the papers' method).
- **MRV backtracking** — the classic solver popularised by Norvig: propagate, then branch on the cell with the fewest candidates.
- **Dancing Links** — Knuth's Algorithm X on the exact-cover form of Sudoku, choosing the column with the fewest rows.
- For the other puzzles: a **DP line solver** (Nonograms), **cage-aware MRV backtracking** (Killer, KenKen), **row-by-row backtracking** (Star Battle, N-Queens) and **DSATUR** (maps).

5.1 Sudoku 9×9: fewer givens, harder puzzles

500 unique puzzles were made with seed 2026: 50 at each number of givens from 22 to 40. All six methods solved all 500 correctly.

500 / 500

solved by option layers with
no guess

463 / 500

with pairs only; the group
check was needed in 37

1,186

average MRV search nodes
at 22 givens

1.5 ms

median option-layer time at
22 givens

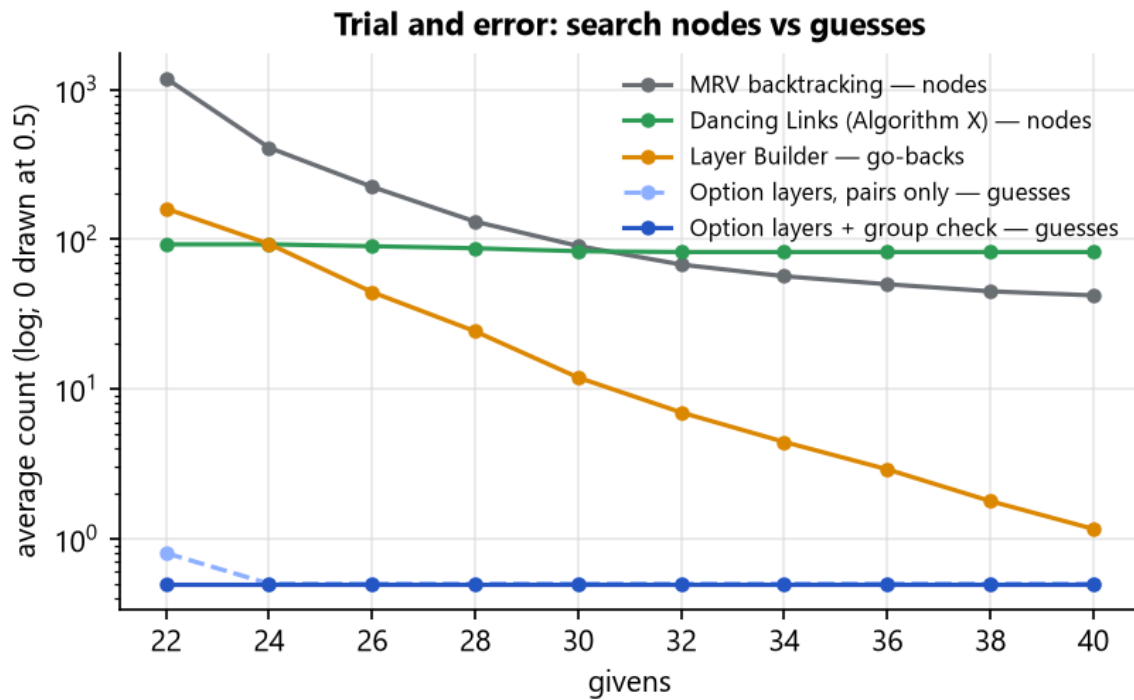


Figure 5.1. Trial and error. MRV backtracking and Dancing Links count search nodes; the Layer Builder counts go-backs; option layers count guesses. Option layers made no guess at any number of givens (0 is drawn at 0.5 on the log scale). Without the group check, a few guesses appear at 22–28 givens.

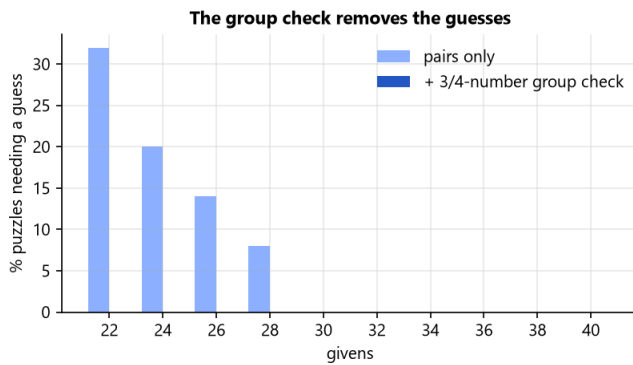


Figure 5.2. Share of puzzles that needed a guess. The 3/4-number group check removes all of them.

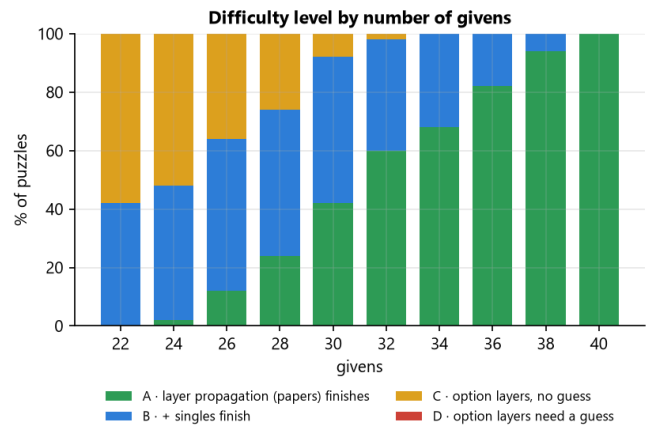


Figure 5.3. The first rule that finished each puzzle. At 22 givens none finish with layer propagation alone, 21 need singles, and 29 need option layers — but no puzzle needs a guess (level D is empty).

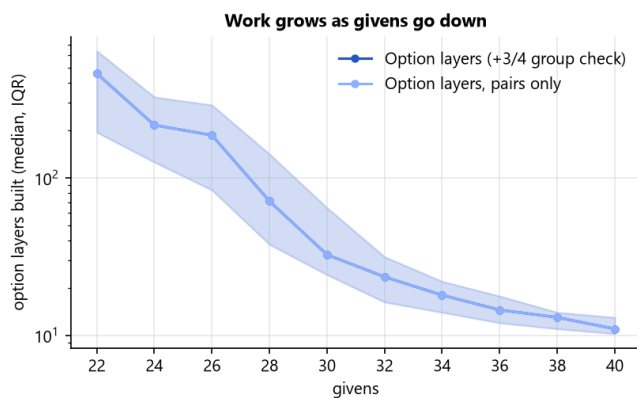


Figure 5.4. How many option lists were built before the puzzle was solved (median and middle half). Thinnest-first building keeps the work small: about 11 at 40 givens, about 460 at 22.

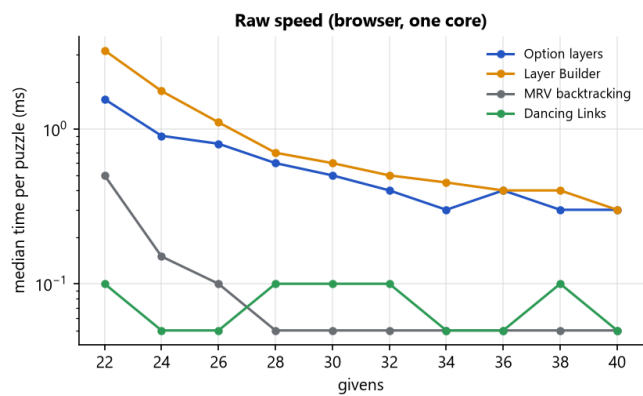


Figure 5.5. Raw speed. Dancing Links is the fastest (about 0.1 ms). Option layers take 0.3–1.5 ms, faster than the Layer Builder at every level. Points at 0.05 ms are medians below the 0.1 ms timer step.

5.2 Larger boards

Hard puzzles (fewest givens that stay unique) were made for 4×4, 6×6, 9×9, 12×12 and 16×16 boards, with rectangular boxes where needed (2×3 for 6×6, 3×4 for 12×12).

Board	puzzles	no guess	option lists built (median)	option layers ms	MRV ms	MRV nodes (avg)	Dancing Links ms
4×4	40	40	4	0.1	<0.1	13	<0.1
6×6	40	40	7	0.2	<0.1	25	<0.1
9×9	40	40	83	0.65	0.1	159	0.1
12×12	25	24	1,158	5.7	0.9	1,673	0.1
16×16	12	12	4,769	23.8	1.9	2,064	0.3

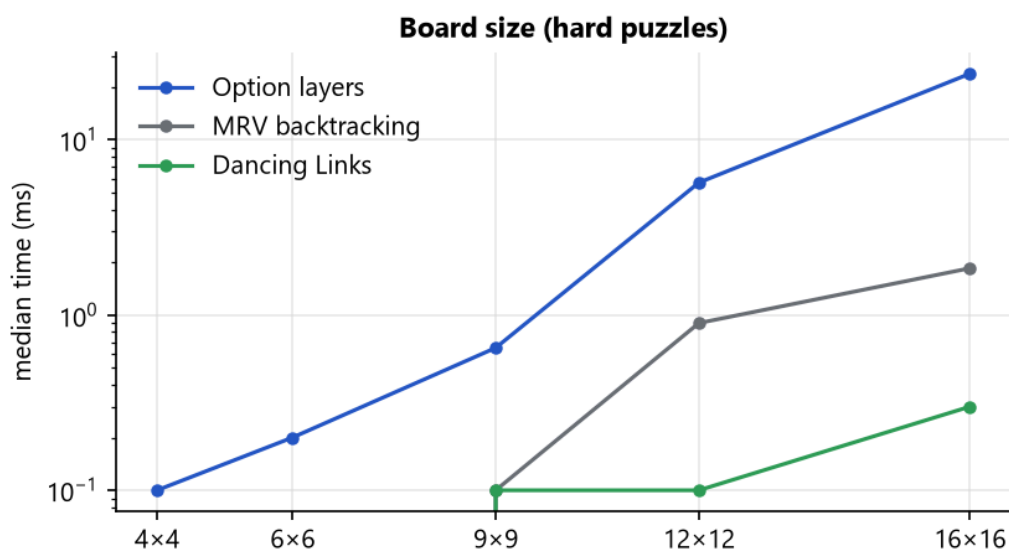


Figure 5.6. Median time by board size. One 12×12 puzzle needed 3 guesses; all other puzzles were solved without a guess. The classic solvers are faster in raw time; their search trees grow with size while option layers stay guess-free.

5.3 All puzzle types

Each puzzle page's own generator made unique puzzles with fixed seeds. The same option-layer engine that the page shows step by step was timed against a classic solver for that puzzle.

Puzzle	Sizes	puzzles	no guess	Baseline	Note
Sudoku 9×9	22–40 givens	500	100%	Dancing Links	group check used in 37
Latin squares	5, 7, 9	105	100%	MRV	fewest givens that stay unique
Killer Sudoku	6×6, 9×9	60	100%	cage-aware MRV	9×9: 29 cages, median 46,172 search nodes for MRV
KenKen	5, 6, 7	80	100%	cage-aware MRV	+ − × ÷ cages
Star Battle	6×6 1★, 8×8 1★, 10×10 2★	71	97%	row backtracking	probe used in every puzzle
N-Queens	8 to 30, fewest givens	105	85%	row backtracking	option layers solved all 105 ; backtracking hit its 5-million-node cap on 5 boards (20×20 and 30×30)
Nonograms	10 to 20, three densities	145	100%	DP line solver	line solver needed guesses on 16 puzzles

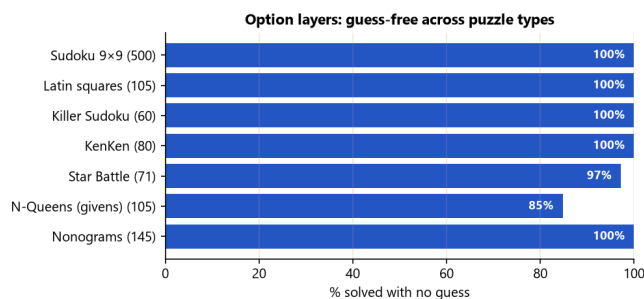


Figure 5.7. Share of puzzles solved without any guess.

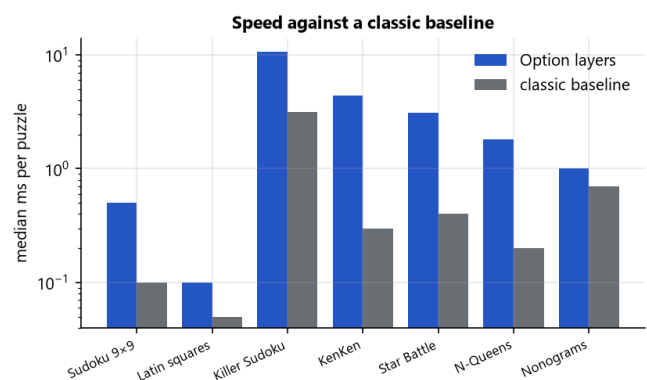


Figure 5.8. Median time per puzzle (log scale). Bars at 0.05 ms are below the 0.1 ms timer step.

Where option layers win. Two results stand out. First, **N-Queens with givens**: at 30×30 the option layers took a median of 29 ms, while row backtracking took a median of 430 ms and failed to finish 4 of 10 boards within 5 million nodes. The attack rule empties whole diagonals at once, and locking a row with one cell left spreads far. Second, **sparse Nonograms**: a classic line solver settles each line on its own and then must guess. On sparse 15×15 puzzles it guessed in 14 of 25 (1,341 guesses in total). Option layers, with the "all lines together" probe, solved every one without a guess (Figure 5.9).

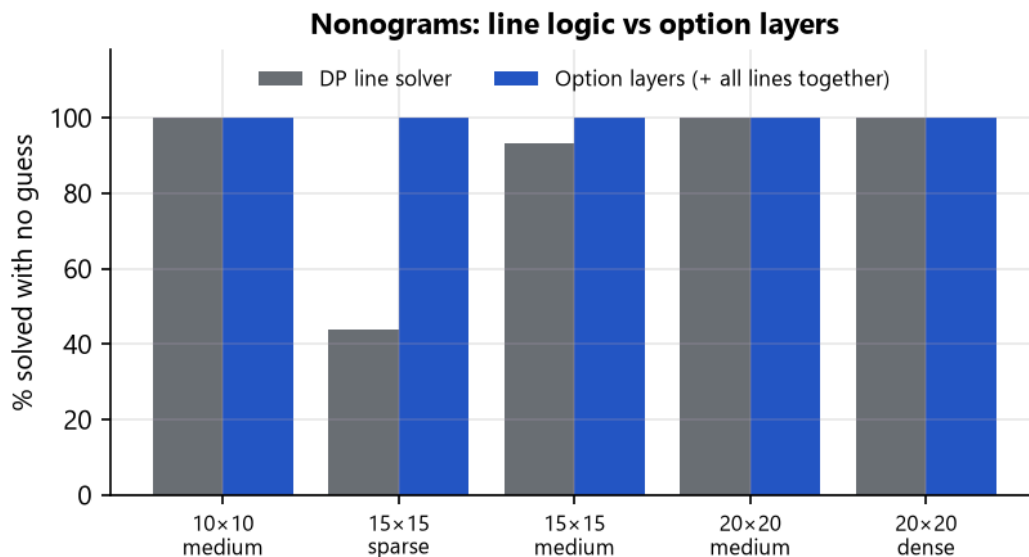


Figure 5.9. Nonograms solved with no guess, by size and density (share of black cells).

Where classic search wins. For small, dense puzzles (KenKen, Star Battle 1★, Latin squares), simple backtracking finds the answer in well under a millisecond, before option lists would even be built. Option layers pay for listing whole placements up front; that cost is repaid only when the search tree would otherwise be large. This is the engine written for the pages, with its step log; Section 5.4 shows what the core rules cost on their own.

5.4 Lean option layers: only build, cut and AND

The option-layer engine used so far records every step for the pages and adds the pair check, the 3/4-number group check and the probe. To see what the *core* of the method costs, a second engine was written with only four rules:

1. **Build** — start with only the top 2 (or 3) numbers, chosen as the ones with the fewest open cells (or the most repeated); list their options. After the results below, fewest open cells became the default in the Speed page. Add the next number only when nothing else changes. Layers are never all built at once.
2. **Cut** — remove options that use a cell the number can no longer take.
3. **AND / OR** — a cell in every option is certain; a cell in no option is removed; one option left = locked.
4. **Pillar** — a cell that only one number can still reach gets that number.

A guess (fewest options first) is made only when every layer is built and nothing changes. The data is kept in bit sets, so a cut or an AND is a few machine-word operations. There is no step log, no pair or group check, and no probe. The engine is in the new page **LayerSudoku-Speed.html**; short solves are repeated and averaged so that times below the 0.1 ms timer step are still measured.

Two engines, two jobs

The full engine is for **explaining**: no guesses and a reason for every step. The lean engine is for **speed**: the same layers, cuts and ANDs, at the speed of the fastest classic solvers.

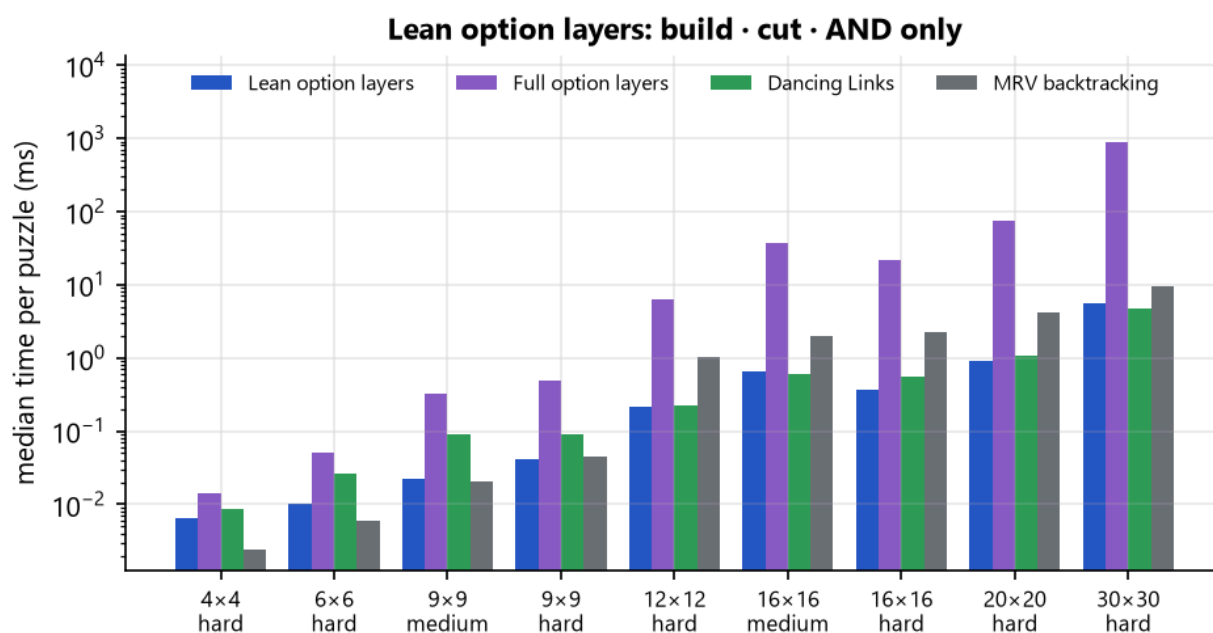


Figure 5.10. Median time per puzzle for the same puzzles (log scale). 4×4, 6×6 and 9×9: 100 puzzles each; 12×12: 30; 16×16: 10 of each level; 20×20: 20; 30×30: 10. Every method solved every puzzle correctly.

Board	puzzles	givens	lean, most repeated first	lean, fewest open cells first	full option layers	Dancing Links	MRV	lean guessed	lean layers built
4×4 hard	100	4.5	0.0065	0.0057	0.014	0.0085	0.0024	0	2.0 of 4
6×6 hard	100	11.5	0.010	0.0097	0.050	0.026	0.0060	1	2.3 of 6
9×9 medium	100	31.7	0.022	0.020	0.33	0.089	0.021	4	3.9 of 9
9×9 hard	100	27.5	0.042	0.037	0.49	0.091	0.046	23	5.9 of 9
12×12 hard	30	54.1	0.22	0.21	6.3	0.22	1.05	18	11.4 of 12
16×16 medium	10	110.8	0.64	0.50	36	0.60	2.0	5	15.4 of 16
16×16 hard	10	110.2	0.38	0.36	21	0.56	2.3	2	13.6 of 16
20×20 hard	20	189.8	0.92	0.73	74	1.07	4.1	1	15.1 of 20
30×30 hard	10	480.0	5.6	3.4	879	4.8	9.5	0	19.6 of 30

Median milliseconds per puzzle; green = fastest. Both lean columns start with the top 2 layers. "Lean guessed" and "layers built" are for the most-repeated start. On 16×16 and larger boards, removing givens stops near half the cells (about 43% at 16×16, 48% at 20×20, 53% at 30×30), because proving that each puzzle still has one answer gets too slow; so these puzzles are easier than hard 9×9 ones, and "medium" and "hard" give the same number of givens at 16×16.

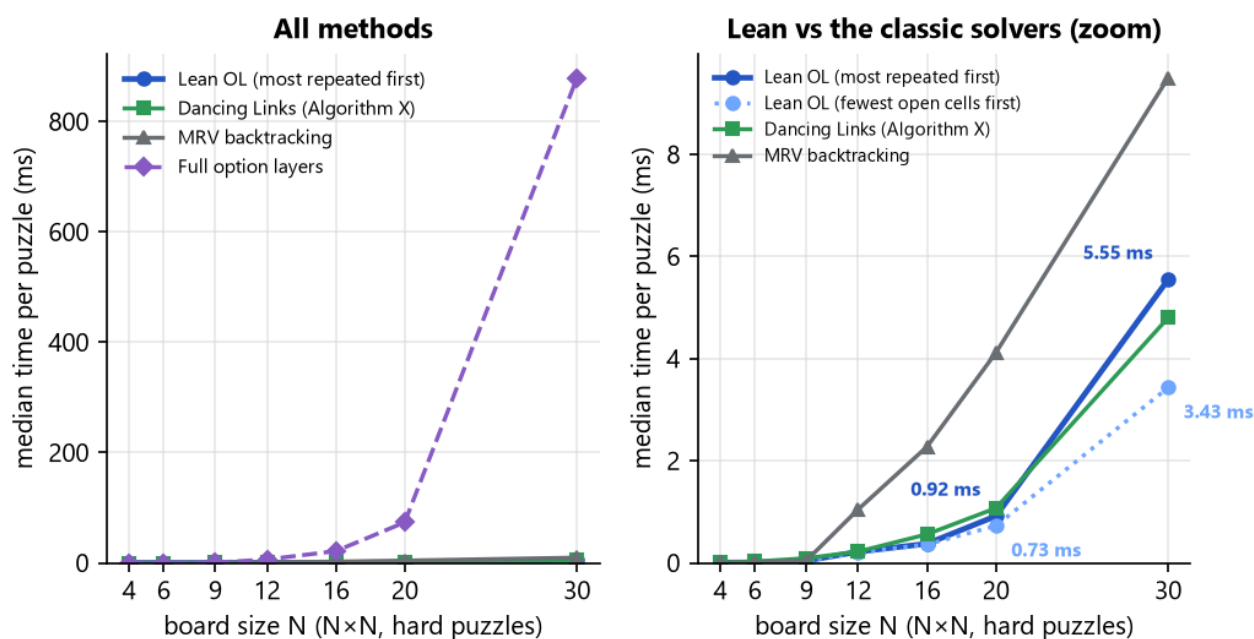


Figure 5.11. The same medians on a normal (linear) scale, hard puzzles, by board size N . Left: with the full engine, whose time climbs steeply (about 0.9 s per puzzle at 30×30). Right: zoomed in on the two lean versions and the classic solvers. MRV backtracking grows fastest (9.5 ms at 30×30). Lean (most repeated first) and Dancing Links stay close (5.6 and 4.8 ms at 30×30). Lean started from the fewest open cells is the lowest line from 9×9 upwards (0.73 ms at 20×20 , 3.4 ms at 30×30).

What the lean test shows.

- **The core is fast, and the gap grows with the board.** Lean option layers were $2\times$ faster than the full engine on 4×4 , $12\text{--}15\times$ on 9×9 , $29\times$ on 12×12 , $57\times$ on 16×16 , $80\times$ on 20×20 and about $160\times$ on 30×30 . The full engine spends its time on the step log, the pair and group checks and on listing more layers (on 30×30 a median of about 59,000 options, against about 900 for lean).
- **It matches or beats the classic solvers.** Started from the numbers with the fewest open cells, lean had the fastest median on every board from 9×9 to 30×30 (level with MRV on 9×9 medium and with Dancing Links on 12×12); at 30×30 it took 3.4 ms against 4.8 ms for Dancing Links and 9.5 ms for MRV. Started from the most repeated number, it was fastest on 9×9 hard, 12×12 , 16×16 hard and 20×20 , while Dancing Links was slightly faster on 16×16 medium (0.60 vs 0.64 ms) and 30×30 (4.8 vs 5.6 ms). Plain MRV wins only on 4×4 and 6×6 , where there is almost nothing to search; from 12×12 up lean was $1.7\text{--}6\times$ faster than MRV.
- **The price is some guessing and a longer tail.** Without the pair and group checks, lean guessed on 23 of 100 hard 9×9 puzzles and 18 of 30 12×12 puzzles (the full engine: none); on the bigger boards, which keep about half of the cells as givens, it rarely guessed (2 of 10 at 16×16 , 1 of 20 at 20×20 , none at 30×30). The guesses are cheap, but they make the slowest puzzles slower: the slowest 20×20 took 14 ms against 1.4 ms for Dancing Links, and the *average* on 12×12 and 20×20 (0.31 and 1.6 ms) was higher than Dancing Links' (0.24 and 1.1 ms).
- **The starting rule matters more on big boards.** On 9×9 and 12×12 , starting with the top 2 or 3 layers, by most repeated or by fewest open cells, changed the median by only about 10%. On 20×20 the fewest-open-cells start was about 20% faster, and on 30×30 about 40% faster (3.4 vs 5.6 ms).

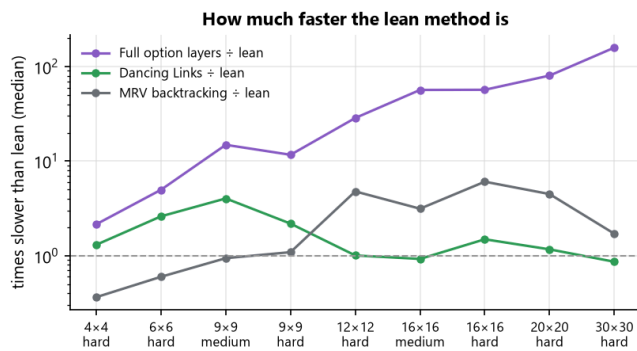


Figure 5.12. How many times slower each method is than the lean one (median). Above the dashed line = lean is faster.

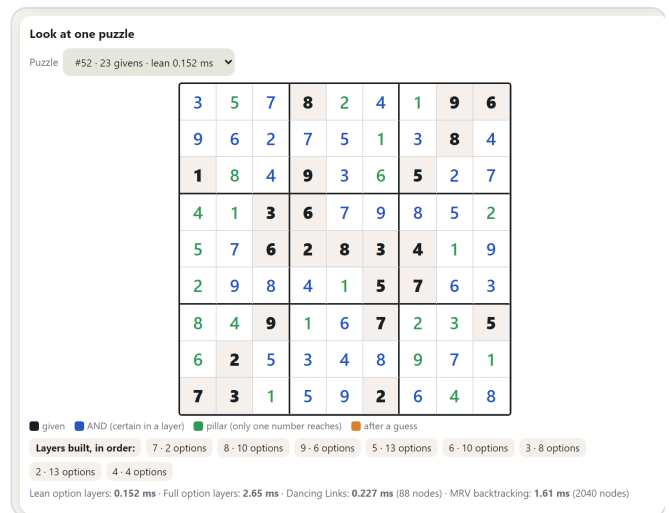


Figure 5.13. A 23-given 9×9 puzzle solved by the lean method with no guess: blue cells came from AND, green from the pillar rule. Only 8 layers were built, with 2–13 options each; 0.15 ms.

Results							
Method	Correct	Median	Average	Slowest	Guessed	Guesses / nodes (avg)	Options built (median)
Lean option layers (build · cut · AND)	100/100	0.104 ms	0.138 ms	1.38 ms	23	0.6	23 (6.8 layers)
Full option layers (+ pairs, 3/4 groups)	100/100	0.993 ms	2.50 ms	42.1 ms	0	0.0	89
Dancing Links (Algorithm X)	100/100	0.207 ms	0.219 ms	0.775 ms	100	90.4	–
MRV backtracking	100/100	0.105 ms	0.212 ms	1.94 ms	100	260.1	–

Options built = total whole placements listed (lean and full option layers). Guessed = puzzles where a guess was needed. MRV = backtracking on the cell with the fewest candidates; Dancing Links = Knuth's Algorithm X on the exact-cover form.

Figure 5.14. The Speed page after 100 hard 9×9 puzzles in the browser: lean has the fastest median (0.104 ms) and is about 10× faster than the full option layers.

6. Optimisation: map colouring and Star Cost

A logic puzzle asks for *one* answer that fits. An optimisation problem asks for the *best* of many answers. The option lists help in both: they say which answers are still possible, and they give cheap limits on how good an answer can be.

6.1 Map colouring: the fewest colours

Maps were made by growing random regions (Voronoi cells) and joining neighbours. The page finds the fewest colours k by trying $k = (\text{largest group of mutual neighbours}), k+1, \dots$ and proving each smaller k impossible. Each try is an option-layer solve: regions are units, colours are options, and a shared border is the pillar. A triangle of neighbouring regions is checked as a 3-unit group.

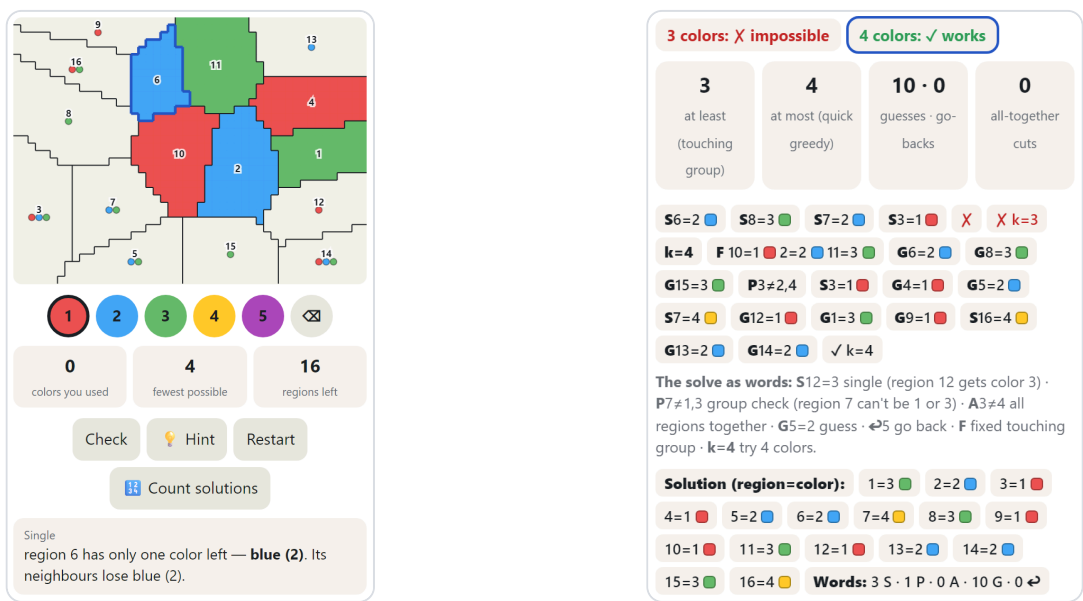


Figure 6.1. Candidate colours as dots during a solve.

Figure 6.2. The "words" of the solve (e.g. $S_{12}=3$, $P_7 \neq 1, 3$, $\leftrightarrow 5$) map each step to the numbered regions.

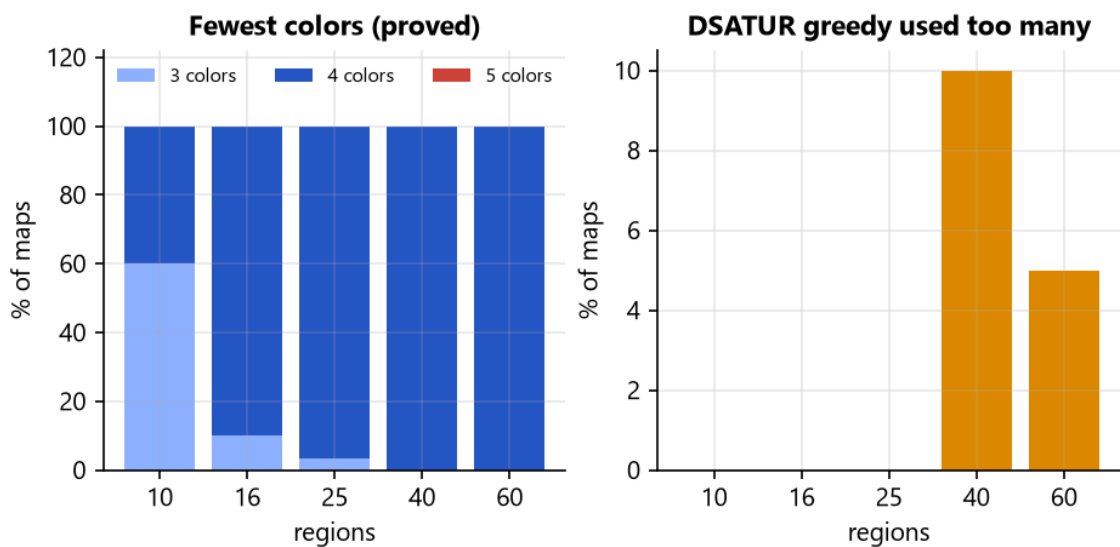


Figure 6.3. 130 maps with 10–60 regions (seed 8000 + regions). Left: proved fewest colours. Right: how often the DSATUR greedy rule used one colour more than needed.

regions	maps	need 3	need 4	DSATUR too many	option layers ms (median)	plain backtracking proof, nodes (avg)
10	30	18	12	0	0.15	21
16	30	3	27	0	0.5	73
25	30	1	29	0	1.5	266
40	20	0	20	2	6.7	3,004
60	20	0	20	1	18.6	172,536

How many colourings work? The page's Count button counts every colouring. On the 10-region maps, only 0.01% (median) to 0.2% (most) of all ways to give each region one of the k colours obey the border rule; on 16-region maps it is about 0.0007%. The answers that work are a tiny island in a huge sea, which is why cutting whole options early pays off. No map needed more than four colours, as the four-colour theorem promises.

6.2 Star Cost: a weights layer on Star Battle

Star Cost adds a **weights layer** on top of a Star Battle board. Every cell has a cost from 1 to 9. The stars must still obey the rules (K stars in every row, column and region, no two touching), and the goal is the placement with the **lowest total cost**. This is like choosing where to put K facilities per area, when neighbouring sites may not both be used.

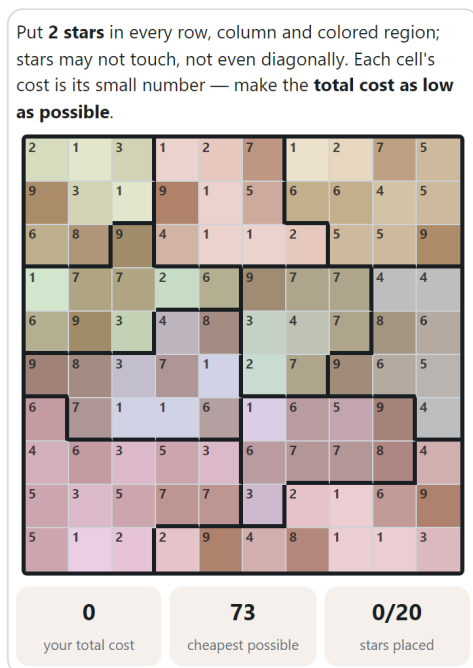


Figure 6.4. A 10×10 board with 2 stars per unit. Darker cells cost more.

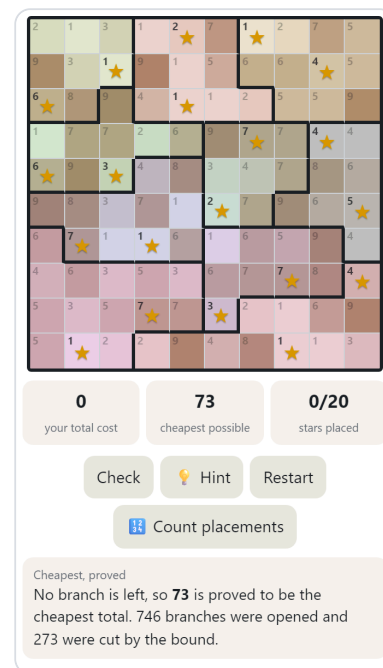


Figure 6.5. The same board solved and proved: the cheapest total is 73. The lower bound began at 50; 746 branches were opened and 273 were cut by the bound.

Exact methods (branch and bound)

- **Simple bound.** For each family of units (rows, columns, regions), add up the cheapest option of every unit; the largest of these three sums is a lower limit on any answer.
- **Weight descent bound.** Put a price on each column and region, fold the prices into the row costs, and take the cheapest row options. Adjust the prices by gradient steps to raise the limit. (This is Lagrangian

relaxation with subgradient steps.)

- **Branching.** Either on the unit with the fewest options, or on the **cheapest free cell** (the author's rule: try the cheapest star first).

Fast heuristics

- ⚡ **Cheapest-first.** Put stars on the lowest weights first, and move to the next cell only when the no-touch rule or a unit count fails; the option layers keep every step legal.
- 🔒 **Lock cheapest (look-ahead m).** Try the m cheapest free cells; for each, finish the board cheapest-first and keep the cell whose finish is lowest. Lock it and repeat.
- ⚙️ **Downhill.** From any answer, apply the best "rectangle swap" (two stars trade columns) until no swap lowers the cost.
- **Unstick.** When downhill stops, free about 60% of the rows (or columns), and rebuild them exactly with the option layers and bounds. Keep the result if it is cheaper. This is large-neighbourhood search.
- For comparison: **random start + downhill** (5 restarts) and **simulated annealing** (20,000 swap steps).

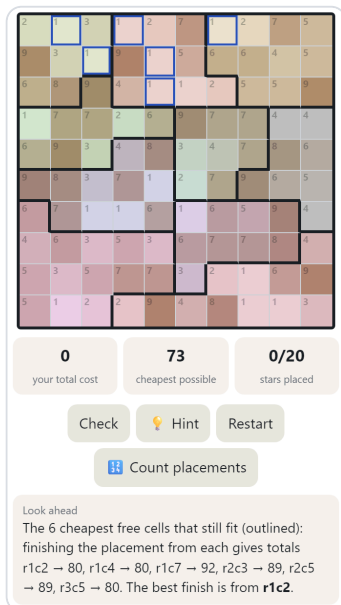


Figure 6.6. Lock cheapest with look-ahead.

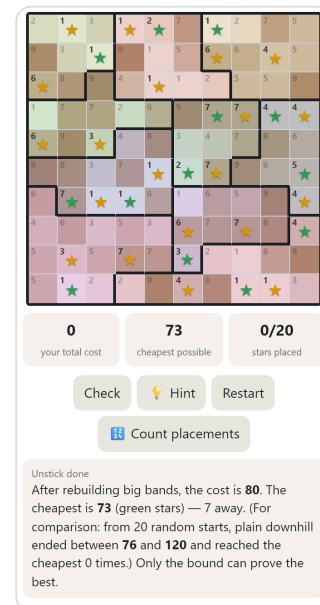


Figure 6.7. Downhill, then Unstick.

Results

58 boards with random weights were solved by every method on the same seeds (Appendix). The exact methods had 8 seconds per board. "Best known" is the lowest cost any method found; it is proved cheapest on 50 of the 58 boards.

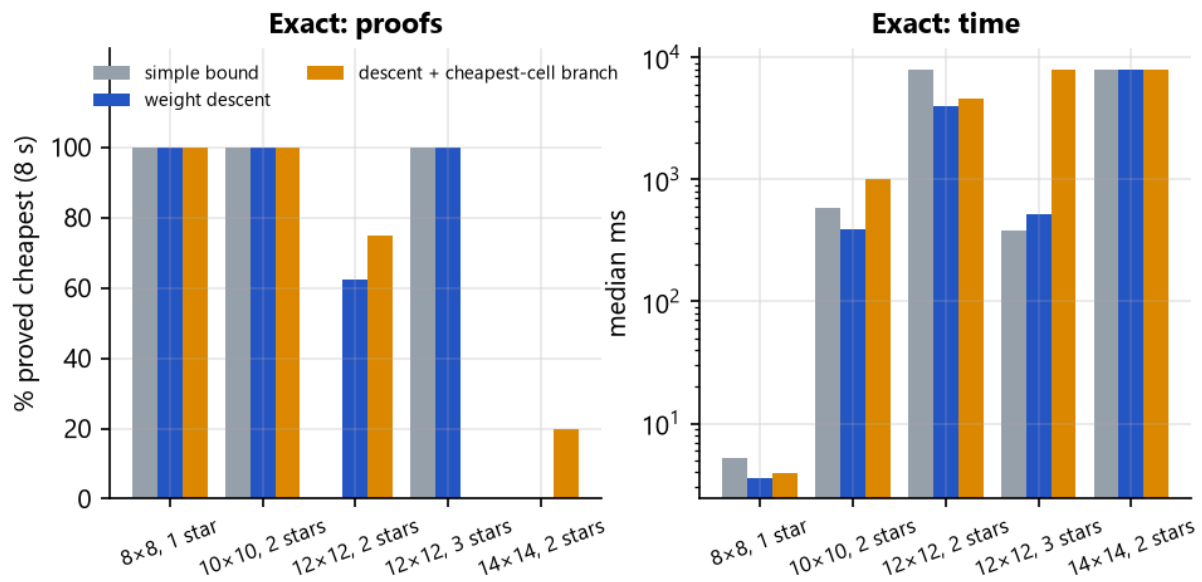


Figure 6.8. Exact branch and bound. Left: boards proved cheapest within 8 s. Right: median time (8,000 ms = ran out of time).

Board	boards	simple bound: proved	descent: proved	descent + cheapest-cell: proved	root gap, simple	root gap, descent
8×8, 1 star	20	20 (5 ms)	20 (4 ms)	20 (4 ms)	36%	11%
10×10, 2 stars	15	15 (590 ms)	15 (388 ms)	15 (1,000 ms)	41%	31%
12×12, 2 stars	8	0	5	6	36%	23%
12×12, 3 stars	8	8 (380 ms)	8 (515 ms)	0	47%	44%
14×14, 2 stars	5	0	0	1	36%	24%

Root gap = how far the first lower bound lies below the best known cost (average). Times are medians.

What the exact results show. The weight-descent bound starts much closer to the answer (11% instead of 36% below on 8×8), so fewer branches are opened. It is the difference between proving 0 and 5 of the 12×12 two-star boards. Branching on the cheapest cell helps on 12×12 two-star boards (6 proved) but fails on three-star boards, where so few placements fit that choosing by unit (fewest options) is far better. The two rules complement each other; the page lets the user pick.

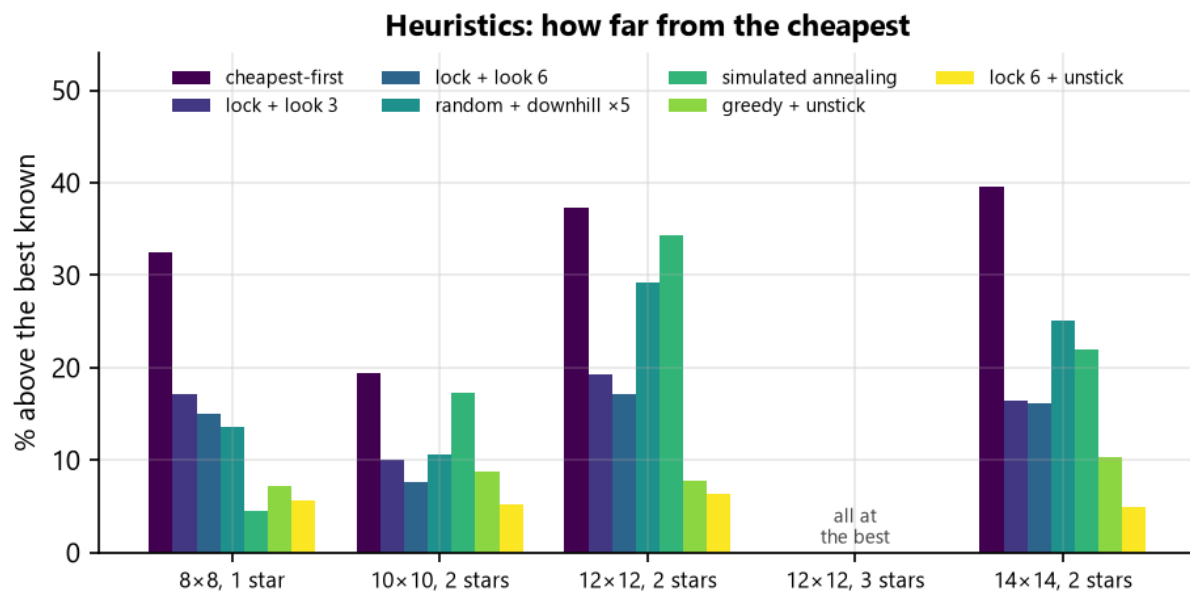


Figure 6.9. Average distance above the best known cost for each heuristic. On 12×12 three-star boards so few placements exist that every method that finished found the best.

Heuristic	8×8, 1★	10×10, 2★	12×12, 2★	14×14, 2★	typical time
⚡ cheapest-first	32.5%	19.5%	37.2%	39.6%	1–225 ms
🔒 lock cheapest, look 3	17.1%	10.1%	19.3%	16.4%	0.01–2 s
🔒 lock cheapest, look 6	15.1%	7.6%	17.1%	16.1%	0.01–3.6 s
random start + downhill ×5	13.7%	10.6%	29.2%	25.1%	3 ms–1 s
simulated annealing	4.4%	17.3%	34.3%	21.9%	50–330 ms
greedy + downhill + Unstick	7.3%	8.8%	7.8%	10.3%	7 ms–1.1 s
lock 6 + Unstick	5.6%	5.2%	6.4%	4.9%	0.02–4.8 s

What the heuristics show. Plain cheapest-first is fast but greedy: it lands 20–40% above the best. Locking the cheapest cell with a look-ahead halves that gap. Unstick — freeing a large band of rows or columns and rebuilding it exactly with option layers — is the strongest single step, and **lock 6 + Unstick** was the best heuristic on every board size except the smallest, within about 5–6% of the best. Simulated annealing is best on small 8×8 boards but falls behind as the board grows, because its small moves rarely get past the no-touch rule. Plain cheapest-first could not finish 6 of the 58 boards within 3 s; the locking methods always finished.

7. Comparison with existing algorithms

The table places each part of the method next to the known technique it is closest to. The full engine is not claimed to beat these algorithms in raw speed; it is a single, explainable framework that covers all of them. The lean engine (Section 5.4) does match them in speed.

Existing algorithm	How it works	Relation to option layers	In our tests
Backtracking with MRV (Norvig; the papers' branching)	Propagate single candidates, then branch on the cell with fewest values.	Option layers keep MRV only as the last rule ("guess last"), and branch on a unit's options, not a cell's values.	Faster than the full engine on 9×9 (0.1–0.5 ms), but up to 1,186 nodes per puzzle at 22 givens. In the lean test MRV was fastest on 4×4 and 6×6; lean was level on 9×9 and 1.7–6× faster from 12×12 to 30×30.
Dancing Links / Algorithm X (Knuth)	Exact cover: choose one row for every column, with fast undo.	Same exact-cover picture: an option is a set of rows of one digit, and a pillar is a cell column. DLX searches row by row; option layers reason with whole sets and cut before searching.	Faster than the full engine on every size (5× on 9×9, about 180× on 30×30). Lean option layers were 2–4× faster up to 9×9 and level on 12×12–20×20; at 30×30 Dancing Links beat the most-repeated start (4.8 vs 5.6 ms) but not the fewest-open-cells start (3.4 ms).
Pattern overlay / templates (Sudoku solving community)	List all templates of a digit and overlay them to find certain and impossible cells.	The closest known relative: the AND/narrow rule on one layer. Option layers add thinnest-first building, the pair and 3/4-number group checks, locks, probes, and the same idea for seven other puzzles.	Pairs-only (closest to plain overlay) still needed guesses on 37 of 500 puzzles.
Human-style rules (X-Wing, Swordfish, naked/hidden sets)	A catalogue of named patterns.	Single-digit fish patterns are special cases of the AND/narrow rule. No catalogue is needed.	—
Nonogram line solvers (dynamic programming)	Settle each line alone by DP over block positions, repeat, then guess.	A line's options are its block placements; the DP line solver computes their AND and OR. The probe adds "all lines together".	DP guessed on 16 of 145 puzzles; option layers on none.
DSATUR (Brélaz) and graph-colouring branch and bound	Colour the most constrained region first; branch to prove the minimum.	Regions are units, colours are options, borders are pillars; the triangle group check is a 3-unit group check.	DSATUR used one colour too many on 3 of 40 large maps; option layers proved the minimum on all 130.
Branch and bound with Lagrangian relaxation	Relax hard rules with prices, and adjust the prices by subgradient steps to raise the lower bound.	The Star Cost "weight descent" bound is exactly this: prices on columns and regions, updated by gradient steps.	See Section 6.2.
Local search: simulated annealing, steepest descent, large-	Improve a full answer by small moves, or by freeing part of it and rebuilding.	Downhill uses rectangle swaps; Unstick is LNS with an exact option-layer rebuild of 60% of the rows or columns.	See Section 6.2.

7.1 What is new here

- **One vocabulary for many puzzles.** Units, options and pillars describe Sudoku, Latin squares, Killer, KenKen, Nonograms, Star Battle, N-Queens, map colouring and a weighted optimisation problem. The rules and the step-by-step display are shared.
- **Thinnest-first incremental building** of option lists, so that only the needed layers are ever listed (Figure 5.4).
- **Group checks on 3 or 4 units**, which in our tests removed every remaining guess on 9×9 Sudoku.
- **Explainable, guess-free solving:** every step before a guess is a checkable reason, and the hint uses only these steps.
- **A lean engine with the speed of classic search:** build (top 2 layers first), cut and AND on bit sets is enough to match Dancing Links and MRV on boards from 9×9 up to 30×30 .
- **From solving to optimising:** the same option lists give lower bounds and moves for a weighted problem (Star Cost), with cheapest-first, lock-cheapest with look-ahead, and Unstick.

8. When option layers win and when MRV wins

The Few Givens page races the methods on the same puzzles made by filling a random fraction of a solved grid. Few givens usually leave many answers, so these tests show a side of the problem that carved puzzles (exactly one answer) do not. The results below are from 9×9 unless the size is named.

The short rule

What decides the winner is not the fill %, and not only the number of answers. It is **how many wrong choices still look right to simple candidate counting**.

- **MRV wins when the answer is easy to see:** a huge number of answers (almost any choice works) or one answer that plain counting finds. MRV has nothing to build, so its low start-up cost wins.
- **Option layers win when wrong choices look right:** one or a few answers on hard puzzles, and the middle zone on large boards. The layers see hidden reasons (a number's whole placement) that MRV cannot, so they skip the long wrong paths.

8.1 By number of answers

Wasted tries are the MRV guesses that were later undone. The puzzles are 9×9, grouped by their number of answers.

Puzzles	Answers	MRV wasted (median)	Worst	Best method
Random fill, very open	> 1,000	9	13,329	MRV usually (any choice works), but it has rare very long runs
Random fill	101 – 1,000	11	754	Middle zone: MRV wastes the most here; option layers are safer on big boards
Random fill	11 – 100	2	793	MRV on 9×9
Random fill	2 – 10	0	234	MRV on 9×9 (easy for both)
Random fill	exactly 1	0	23	MRV on 9×9 (easy for both)
Carved hard	exactly 1	40	1,588	Option layers (lean), with the gap growing from 12×12 up; on 9×9 lean and MRV are close

The last two rows have the same number of answers (one) but very different MRV work: 0 against 40 wasted tries at the median. A carved puzzle is made by removing givens for as long as the answer stays unique, so it ends with the fewest givens that still force one answer, and its reasons are hidden. A random-fill puzzle with one answer usually has more givens than it needs, and plain counting sees the answer.

8.2 By fill % (random fill)

Fill	Usual answers	MRV wasted (median)	Best method
One given per number, 5 – 20%	thousands and more	0 – 2	MRV (rare huge runs at 20 – 25%, up to 4.4 million steps)
25 – 34%	hundreds (34%: median about 175)	5 – 10	The hardest zone for MRV. On 16×16 at 34 – 40%, option layers solved 30 of 30 while MRV and Dancing Links stopped at their 5-million-step limit.
40%	a few (5% unique)	2	MRV on 9×9, option layers on 16×16
45 – 50% and more	1 to a few (40% unique at 50%, 87% at 70%)	0	Easy for both; MRV is quicker on 9×9 because it has no layers to build

This is the *easy-hard-easy* pattern known from constraint problems (Cheeseman, Kanefsky and Taylor, 1991): with very few givens nearly every choice leads to an answer, with many givens simple counting finds the answer, and in between there are many choices that look right but fail later.

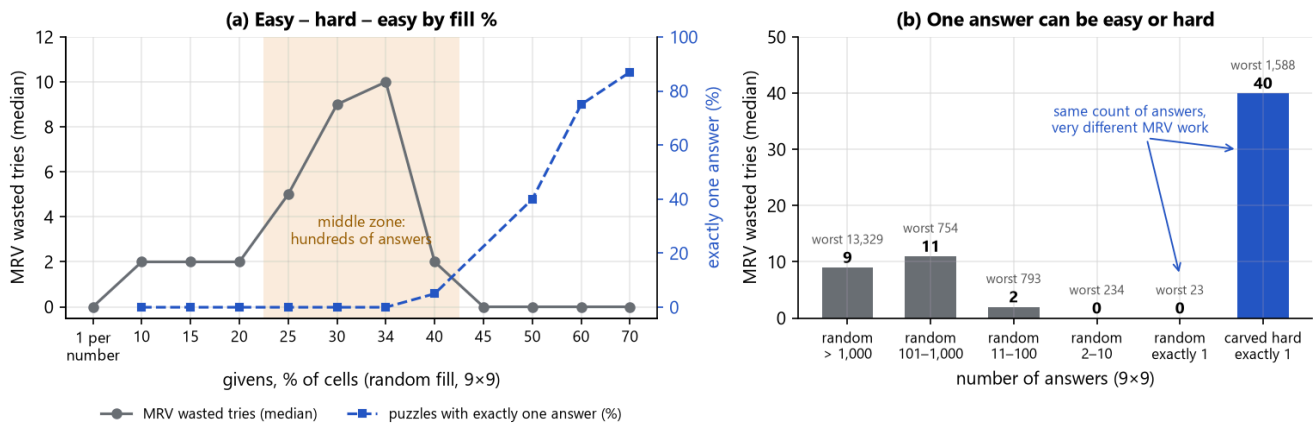


Figure 8.1. Where MRV wastes work, 9×9. (a) By fill %: MRV's wasted tries (grey, left axis) rise to a peak at 30 – 34% and fall to zero from 45%, while the share of puzzles with exactly one answer (blue, right axis) only starts to rise at 40%. The peak is the middle zone with hundreds of answers. (b) By number of answers: random-fill puzzles with one answer cost MRV nothing, but carved hard puzzles with one answer cost it 40 wasted tries at the median. The number of answers alone does not decide the work; how many wrong choices simple counting cannot rule out does.

8.3 By board size (carved hard puzzles, exactly one answer)

Size	Winner	Why
4×4, 6×6	MRV	Almost nothing to search
9×9	Close: lean had the fastest median, MRV level on medium puzzles	Building the layers costs about as much as it saves
12×12, 16×16	Option layers (1.7 – 6× faster than MRV; level with Dancing Links)	Wrong MRV paths grow faster than the cost of building layers
20×20, 30×30, 100×100	Option layers , with a growing gap	Section 5.4, Figures 5.10 – 5.11

8.4 Summary

- **Best for MRV:** small boards, very few givens (many thousands of answers), or many givens (the answer is easy to see).
- **Best for option layers:** large boards (12×12 and up), hard carved puzzles with exactly one answer, and the middle zone of a few hundred answers (about 25 – 40% fill), where MRV and Dancing Links can run for millions of steps.
- **Fill % is only a proxy.** The same fill can give an easy puzzle or a hard one. The number of answers is a better guide, and the best guide is how many wrong choices simple counting cannot rule out.

Limits: the table by number of answers is 9×9 only. For 16×16 and larger there are the 34 – 40% random-fill runs (30 puzzles) and the carved runs of Section 5.4, but no split by number of answers. Worst-case values come from single runs and are less stable than the medians.

9. The cube as a tensor: three more engines

The option-layer cube (row × column × number) can be held as one $N \times N \times N$ true/false **tensor**: a point is true while that number can still go in that cell. Every rule of Sudoku says "exactly one true point on each line of the cube" — each cell, and each number in each row, column and box. This chapter tests three engines built on that view, on the Tensor Layers page ([LayerSudoku-Tensor.html](#)), against the lean engine and the classic solvers.

9.1 Three engines

- **Tensor OL.** The tensor is kept in four views at once (cell, row, column, box), each line packed into one 32-bit word, so a line with no point (a dead end) or one point (a naked or hidden single) is found in one pass. Layers can be cut in four directions: a **number** layer (a horizontal slice: the templates of the lean method), or a **row**, **column** or **box** layer (vertical slices: every way to give its open cells different numbers). The thinnest layer in any allowed direction is built next — top 2 at the start, one more when stuck, never all; a layer over the cap (2,000 options) waits until its size bound shrinks 8 times. Cut and AND / OR are the same in every direction: row, column and box layers find naked and hidden sets, number layers find fish patterns.
- **Matrix OL (matmul).** The tensor is one flat 0/1 vector $\mathbf{x}$. The line sums are one product $\mathbf{A} \cdot \mathbf{x}$ with the fixed line matrix $\mathbf{A}$. All layers of one direction are stacked into one option × point matrix $\mathbf{M}$; one pass is **alive** = $(\mathbf{M} \cdot (\mathbf{1} - \mathbf{x}) = \mathbf{0})$ (cut) and **count** = $\mathbf{alive}^T \cdot \mathbf{M}$ (count 0 → remove, OR; count = the live options of its layer → place, AND). Every row, column and box layer is built **in full** at the start; only the top 2 or 3 number layers are built.
- **Shared number patterns.** A template is a pattern — one cell per row, column and box — that does not depend on the number. So the thinnest numbers are built in **one** walk over the patterns, each pattern kept once with a mask of the numbers it fits. Instead of cutting the lists again, each point keeps a **support count** (the live patterns that use it) and each number its live count; removing a point touches only the patterns through that cell. Support 0 removes a point; support equal to the live count places it. After a failed guess the changes are undone from a trail, so the list is never scanned or copied again. When stuck, one more number joins the shared set.

9.2 Results

10 seeded puzzles per case, the same puzzles for every method, after a warm-up; fast runs are repeated and averaged. Tensor and shared engines: start top 2, cap 2,000. Median time per puzzle in ms; **green** = fastest overall, **bold** = fastest option-layer engine where a classic solver was faster; a red fraction = puzzles solved within the 5-million-step limit.

Puzzles	Givens	Tensor · numbers	Tensor · all 4	Tensor · R/C/B	Shared patterns	Matrix top 2	Matrix top 3	Lean OL	Dancing Links	MRV
9×9 hard, 1 answer	28	80 μs	0.23	0.17	0.13	0.69	0.66	94 μs	0.18	56 μs
9×9 random 20%	16	8.73	20.7	16.2	4.54	60.0	49.8	4.71	0.20	78 μs
9×9 random 34%	28	0.44	1.42	0.80	1.22	1.60	1.57	0.24	0.24	54 μs
16×16 hard, 1 answer	108	0.69	0.50	0.54	0.37	16.0	13.5	1.08	1.26	3.73
16×16 random 40%	102	21.3	58.8	39.0	15.0	not run	not run	47.4	1.40	255 (8/10)

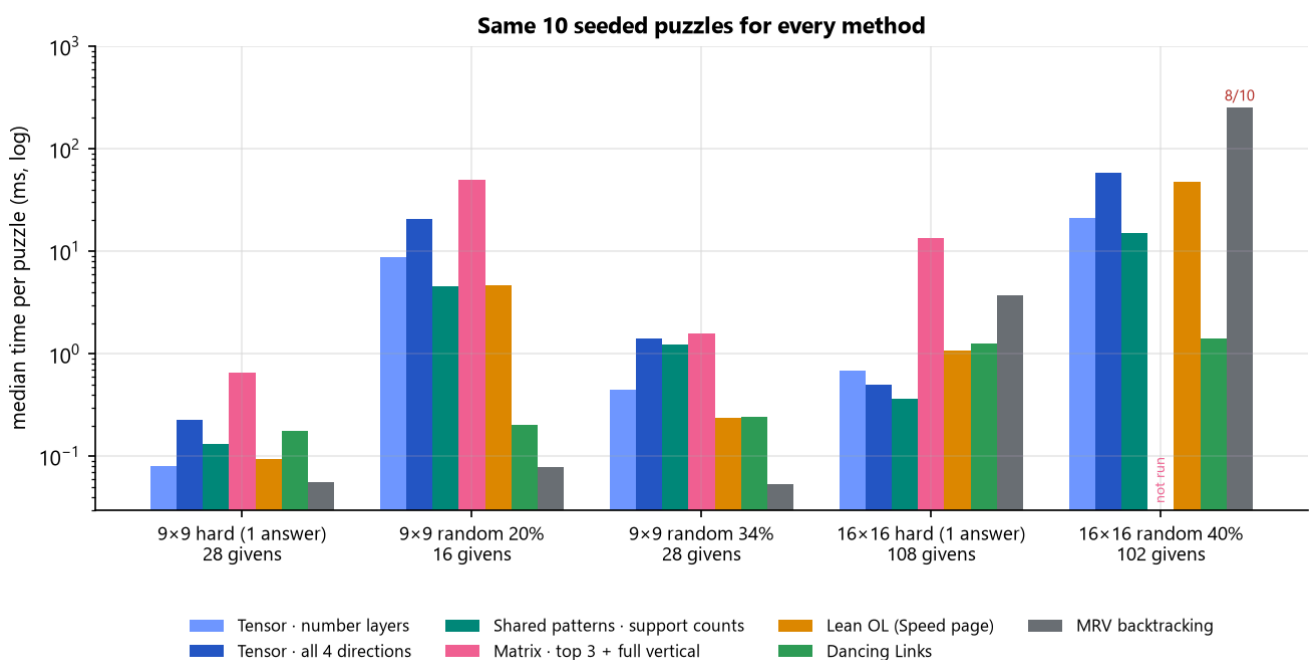


Figure 9.1. The same medians on a log scale. The matrix engines were not run on 16×16 random 40%: building every row, column and box layer in full is too large on an open 16×16 board. MRV solved 8 of 10 there within its 5-million-step limit.

9.3 What the results show

- **16×16 hard (one answer): option-layer engines are fastest.** Shared patterns 0.37 ms and tensor (all directions) 0.50 ms, against lean 1.08, Dancing Links 1.26 and MRV 3.73 ms.
- **Shared patterns is the best option-layer engine on the most open boards.** 9×9 random 20%: 4.54 ms against 8.73 ms for number layers built one at a time and 4.71 for lean; 16×16 random 40%: 15.0 ms against 21.3 (tensor) and 47.4 (lean). Building several numbers in one walk, and using support counts instead of cutting the lists again, pay off where the lists are large and overlap. At 9×9 random 34% it does not: 1.22 ms against 0.24 for lean and 0.44 for tensor number layers — the lists there are small, and setting up the counts costs more than it saves.
- **Classic search still wins on small or open boards.** On 9×9, MRV is fastest in every case (56 μs on hard puzzles): with many answers almost any guess works, and MRV guesses cheaply. On 16×16 random 40%, Dancing Links (1.40 ms) is about ten times faster than any option-layer engine — while MRV (255 ms, 2 of 10 unsolved) is slower than all of them.

- **More layers are not always better.** Tensor with all four directions is slower than number layers alone on 9×9 and on open 16×16 : it builds more layers than it needs. Building the vertical layers in full (the matrix engines) is the slowest option everywhere — on 16×16 hard the full lists hold 6,000–21,000 options, and every pass multiplies all of them.
- **The matmul form fits a GPU, not a CPU.** Its two products are what graphics hardware does well; on the CPU they cost more than they save. A GPU version would only pay on large boards, where the start-up cost is small next to the work.

Limits: 10 puzzles per case, one machine, Node (V8) rather than the browser; sizes above 16×16 and other fills were not measured. Script: `report/tools/exp_tensor.mjs`; data: `report/data/tensor.json`; chart: `report/tools/f_tensor.py`.

10. Discussion, limits and related work

10.1 What the experiments show

- **Guess-free solving is the main strength.** Across 1,066 logic puzzles of seven types, option layers solved 1,048 (98%) with no guess at all, and all of them correctly. All 500 9×9 Sudoku, including the hardest at 22 givens, needed no guess.
- **Group checks matter.** Pairs alone left 37 of 500 Sudoku unfinished; the 3/4-number group check finished all of them.
- **Option layers are strongest when classic search is weakest:** N-Queens with givens at 20×20 and 30×30, and sparse Nonograms.
- **Speed depends on the engine, not the idea.** The full engine, written for display, is about 5× slower than Dancing Links on 9×9 and about 180× slower on 30×30. The lean engine — the same layers, cuts and ANDs on bit sets, with no step log and no pair or group check — had the fastest median on every board from 9×9 to 30×30 when it started from the fewest open cells.
- **Explaining and speed pull in different directions.** The pair and group checks remove every guess but cost time; without them the lean engine guesses on about a quarter of hard 9×9 puzzles and still finishes first.

10.2 Limits

- **Option lists can be large.** A 16×16 digit has millions of templates on an empty board. The thinnest-first rule avoids listing large layers, but a sparse, large board can still be slow (16×16 median 24 ms, with up to several thousand lists built).
- **Star Battle with 2 stars** has many options per unit; the 10×10 2★ puzzles took about 1 s at the median, and up to 9 s.
- **Small numbers of puzzles** at the biggest sizes (12 at 16×16, 10 at 30×30 N-Queens, 11 at 10×10 2★ Star Battle) because making unique hard puzzles is slow. Results there are indicative.
- **Browser timing** has a 0.1 ms step, and all runs used one machine. Only relative times should be read from the charts.
- The group check is exact for the units it looks at, but it does not look at every group of 3 or 4. A puzzle can exist that needs a guess; one 12×12 Sudoku did (3 guesses).

10.3 Related work

The idea of listing whole placements of a digit is known in the Sudoku community as the *pattern overlay method* or *templates*. Exact cover and Dancing Links (Knuth, 2000) give the same set picture for search. Nonogram line solvers compute the overlap of all placements of a line by dynamic programming. Constraint programming uses *table constraints* (lists of allowed tuples) and *singleton arc consistency*, which is close to the probe rule. Lagrangian relaxation for lower bounds and large-neighbourhood search (Shaw, 1998) are standard in operations research. The contribution of this work is to join these ideas in one layered method, with thinnest-first building and group checks, and to show that the same units-options-pillars view carries across eight puzzles and into optimisation, while keeping every step explainable.

10.4 Future work

- Adding a cheap pair check to the lean engine only when it is about to guess, to keep its speed and remove most guesses.
- Choosing which groups of 3–4 units to check by a score, instead of in order.
- The "language" of moves: grading puzzles by the words (rules) they need, as started in the Language and Categories pages.
- More optimisation problems in the same frame: timetables, placing facilities, and routing with forbidden neighbours.

11. Appendix: seeds, settings and how to reproduce

11.1 Tools

All experiments use the author's single-file HTML pages in `C:\Sudoku_shaimaa`. A Node.js script (Node 24) opens each page in headless Microsoft Edge through the Chrome DevTools Protocol, without any extra package. Before a page loads, `Math.random` is replaced by the seeded generator *mulberry32*, so `__seed(n)` makes every puzzle again exactly. Charts are drawn with Python (matplotlib). The scripts are in `report\tools`:

Script	What it does	Output
<code>cdp.mjs</code>	Starts headless Edge, opens a page, runs code, takes screenshots, prints PDF.	—
<code>baselines.js</code>	The classic solvers: MRV backtracking, Dancing Links, row backtracking for Star Battle, DP line solver, map backtracking, cage-aware MRV.	—
<code>exp_sudoku.mjs</code>	Section 5.1: 50 puzzles at each of 22, 24, ..., 40 givens; six methods each.	<code>data/sudoku_givens.json</code>
<code>exp_puzzles.mjs</code>	Sections 5.2, 5.3, 6.1: sizes, Latin, Killer, KenKen, Star Battle, N-Queens, Nonograms, maps.	<code>data/<name>.json</code>
<code>exp_lean.mjs</code>	Section 5.4: lean vs full option layers, Dancing Links and MRV (engines from <code>LayerSudoku-Speed.html</code>).	<code>data/lean.json</code>
<code>exp_cost.mjs</code>	Section 6.2: Star Cost exact methods and heuristics.	<code>data/starcost.json</code>
<code>shots.mjs</code>	Phone-size screenshots (390×844, 2× pixels).	<code>shots/*.png</code>
<code>charts.py</code>	All figures and the numbers quoted in this report.	<code>fig/*.png</code> , <code>data/_numbers.json</code>

11.2 Seeds and settings

Experiment	Seeds	Settings
Sudoku by givens	2026 (one stream for all 500)	9×9, givens 22–40 in steps of 2, 50 each, unique
Sudoku sizes, Latin, Killer, KenKen, Star Battle, Queens, Nonogram, maps	1000+n (sizes), 2000+n (Latin), 3000+n (Killer), 4000+n (KenKen), 5000+n (Star Battle), 6000+n (Queens), 7000+7n+density% (Nonograms), 8000+R (maps)	sizes and counts as in Sections 5–6; baseline caps: 3–5 million nodes, 5 s
Lean speed test	11000 + 10·n (+1 for hard)	sizes 4–30; 100 (4×4–9×9) / 30 (12×12) / 10 (16×16) / 20 (20×20) / 10 (30×30) puzzles; short solves repeated to at least 8 ms and averaged
Star Cost	9000 + 100·N + 10·K + k	random weights 1–9; exact budget 8 s; heuristics 3 s (cheapest-first), 6 s (lock); annealing 20,000 steps; Unstick 12 rounds
Screenshots	3–51, fixed per page	390×844 viewport, device scale 2

11.3 How to reproduce

```
cd C:\Sudoku_shaimaa\report\tools
node exp_sudoku.mjs
node exp_puzzles.mjs          (all sections; or one name, e.g. killer)
node exp_lean.mjs
node exp_cost.mjs
node shots.mjs
python charts.py
```

Timings depend on the computer; counts (guesses, nodes, proofs) depend only on the seeds.